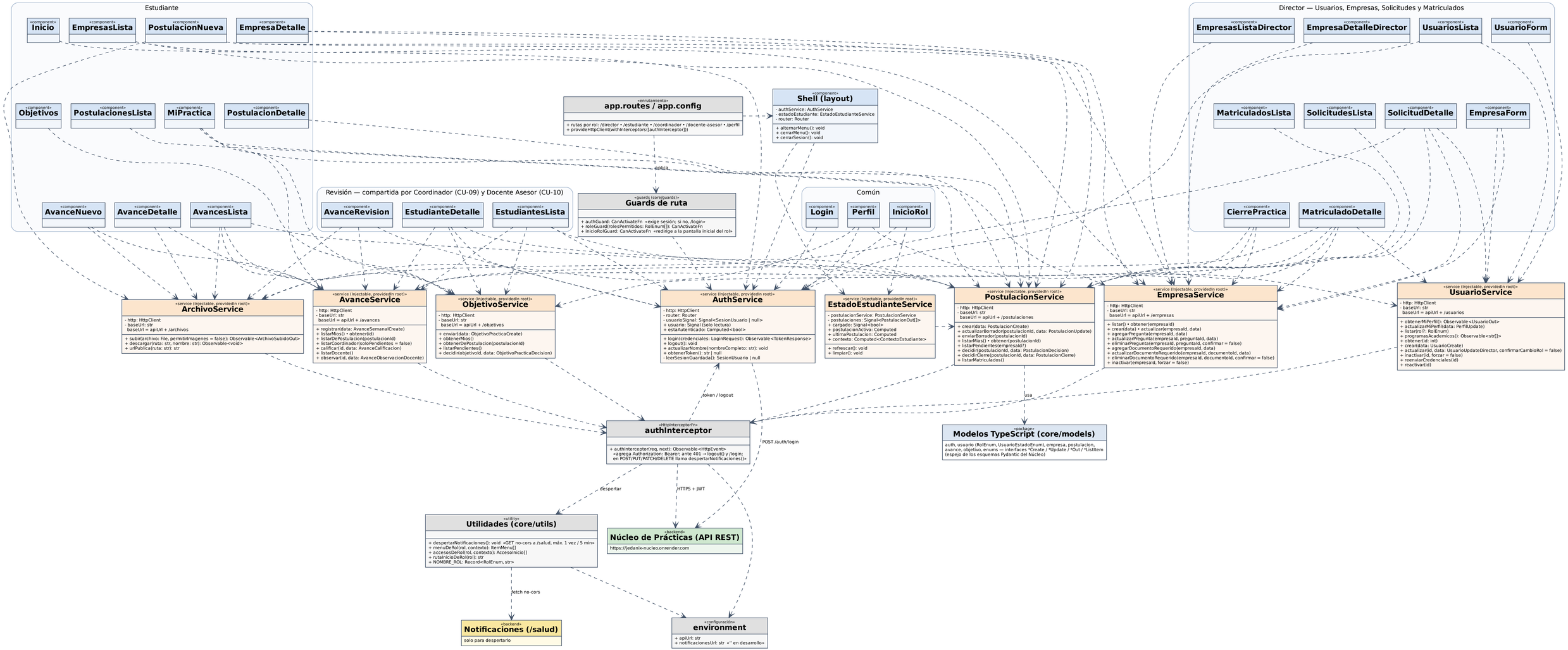
JEDANIX — Diagrama de clases (5/5): Frontend Angular (componentes, servicios, guards e interceptor)
Estudiante
«component» Inicio
«component» EmpresasLista
«component» PostulacionNueva
«component» EmpresaDetalle
«component» Objetivos
«component» PostulacionesLista
«component» MiPractica
«component» PostulacionDetalle
«component» AvanceNuevo
«component» AvanceDetalle
«component» AvancesLista
Director — Usuarios, Empresas, Solicitudes y Matriculados
«component» EmpresasListaDirector
«component» EmpresaDetalleDirector
«component» UsuariosLista
«component» UsuarioForm
«component» MatriculadosLista
«component» SolicitudesLista
«component» SolicitudDetalle
«component» EmpresaForm
«component» CierrePractica
«component» MatriculadoDetalle
«enrutamiento» app.routes / app.config
+ rutas por rol: /director • /estudiante • /coordinador • /docente-asesor • /perfil
+ provideHttpClient(withInterceptors([authInterceptor]))
«component» Shell (layout)
- authService: AuthService
- estadoEstudiante: EstadoEstudianteService
- router: Router
+ alternarMenu(): void
+ cerrarMenu(): void
+ cerrarSesion(): void
aplica
Revisión — compartida por Coordinador (CU-09) y Docente Asesor (CU-10)
«component» AvanceRevision
«component» EstudianteDetalle
«component» EstudiantesLista
«guards (core/guards)» Guards de ruta
+ authGuard: CanActivateFn «exige sesión; si no, /login»
+ roleGuard(rolesPermitidos: RolEnum[]): CanActivateFn
+ inicioRolGuard: CanActivateFn «redirige a la pantalla inicial del rol»
Común
«component» Login
«component» Perfil
«component» InicioRol
«service (Injectable, providedIn root)» ArchivoService
- http: HttpClient
- baseUrl: str
baseUrl = apiUrl + /archivos
+ subir(archivo: File, permitirImagenes = false): Observable<ArchivoSubidoOut>
+ descargar(ruta: str, nombre: str): Observable<void>
+ urlPublica(ruta: str): str
«service (Injectable, providedIn root)» AvanceService
- http: HttpClient
- baseUrl: str
baseUrl = apiUrl + /avances
+ registrar(data: AvanceSemanalCreate)
+ listarMios() • obtener(id)
+ listarDePostulacion(postulacionId)
+ listarCoordinador(soloPendientes = false)
+ calificar(id, data: AvanceCalificacion)
+ listarDocente()
+ observar(id, data: AvanceObservacionDocente)
«service (Injectable, providedIn root)» ObjetivoService
- http: HttpClient
- baseUrl: str
baseUrl = apiUrl + /objetivos
+ enviar(data: ObjetivoPracticaCreate)
+ obtenerMios()
+ obtenerDePostulacion(postulacionId)
+ listarPendientes()
+ decidir(objetivoId, data: ObjetivoPracticaDecision)
«service (Injectable, providedIn root)» AuthService
- http: HttpClient
- router: Router
- usuarioSignal: Signal<SesionUsuario | null>
+ usuario: Signal (solo lectura)
+ estaAutenticado: Computed<bool>
+ login(credenciales: LoginRequest): Observable<TokenResponse>
+ logout(): void
+ actualizarNombre(nombreCompleto: str): void
+ obtenerToken(): str | null
- leerSesionGuardada(): SesionUsuario | null
«service (Injectable, providedIn root)» EstadoEstudianteService
- postulacionService: PostulacionService
- postulaciones: Signal<PostulacionOut[]>
+ cargado: Signal<bool>
+ postulacionActiva: Computed
+ ultimaPostulacion: Computed
+ contexto: Computed<ContextoEstudiante>
+ refrescar(): void
+ limpiar(): void
«service (Injectable, providedIn root)» PostulacionService
- http: HttpClient
- baseUrl: str
baseUrl = apiUrl + /postulaciones
+ crear(data: PostulacionCreate)
+ actualizarBorrador(postulacionId, data: PostulacionUpdate)
+ enviarBorrador(postulacionId)
+ listarMias() • obtener(postulacionId)
+ listarPendientes(empresaId?)
+ decidir(postulacionId, data: PostulacionDecision)
+ decidirCierre(postulacionId, data: PostulacionCierre)
+ listarMatriculados()
«service (Injectable, providedIn root)» EmpresaService
- http: HttpClient
- baseUrl: str
baseUrl = apiUrl + /empresas
+ listar() • obtener(empresaId)
+ crear(data) • actualizar(empresaId, data)
+ agregarPregunta(empresaId, data)
+ actualizarPregunta(empresaId, preguntaId, data)
+ eliminarPregunta(empresaId, preguntaId, confirmar = false)
+ agregarDocumentoRequerido(empresaId, data)
+ actualizarDocumentoRequerido(empresaId, documentoId, data)
+ eliminarDocumentoRequerido(empresaId, documentoId, confirmar = false)
+ inactivar(empresaId, forzar = false)
«service (Injectable, providedIn root)» UsuarioService
- http: HttpClient
- baseUrl: str
baseUrl = apiUrl + /usuarios
+ obtenerMiPerfil(): Observable<UsuarioOut>
+ actualizarMiPerfil(data: PerfilUpdate)
+ listar(rol?: RolEnum)
+ programasAcademicos(): Observable<str[]>
+ obtener(id: int)
+ crear(data: UsuarioCreate)
+ actualizar(id, data: UsuarioUpdateDirector, confirmarCambioRol = false)
+ inactivar(id, forzar = false)
+ reenviarCredenciales(id)
+ reactivar(id)
«HttpInterceptorFn» authInterceptor
+ authInterceptor(req, next): Observable<HttpEvent>
«agrega Authorization: Bearer; ante 401 → logout() y /login;
en POST/PUT/PATCH/DELETE llama despertarNotificaciones()»
/token / logout
POST /auth/login
usa
«package» Modelos TypeScript (core/models)
auth, usuario (RolEnum, UsuarioEstadoEnum), empresa, postulacion, avance, objetivo, enums — interfaces *Create / *Update / *Out / *ListItem (espejo de los esquemas Pydantic del Núcleo)
despertar
HTTPS + JWT
«utility» Utilidades (core/utils)
+ despertarNotificaciones(): void «GET no-cors a /salud, máx. 1 vez / 5 min»
+ menuDeRol(rol, contexto): ItemMenu[]
+ accesosDeRol(rol, contexto): AccesoInicio[]
+ rutaInicioDeRol(rol): str
+ NOMBRE_ROL: Record<RolEnum, str>
«backend» Núcleo de Prácticas (API REST)
https://jedanix-nucleo.onrender.com
fetch no-cors
«backend» Notificaciones (/salud)
solo para despertarlo
«configuración» environment
+ apiUrl: str
+ notificacionesUrl: str «'' en desarrollo»